

Unit 1 : User Defined Functions	1	Demonstrate the user-defined function in C. Write the general form (syntax) of a C function and list its main components.
	2	Outline the need for user-defined functions in C programming and their role in improving program modularity and readability.
	3	Illustrate a C program using a function that accepts two integer arguments and returns their sum, showing the function call from main().
	4	Analyze the working of recursion in C by explaining how a recursive function executes using the example of calculating the factorial of a number.
	5	Evaluate the advantages and disadvantages of using recursion over iteration in C. Give suitable examples to justify your answer.
	6	Design a scenario (with logic or flow explanation) that demonstrates the difference between local and global variables in terms of their scope, visibility, and lifetime within multiple functions.
Unit 02 : Structures & Unions	7	Define a structure in C. Write the syntax for structure definition and list any two features of structures.
	8	Explain how values are assigned to structure members. Illustrate with a suitable example.
	9	Write a C program to initialize a structure and store details of multiple students using an array of structures.
	10	Analyze the difference between arrays within structures and structures within structures with appropriate examples.
	11	Compare Structure and Union in C with respect to memory allocation, access to members, and usage.
	12	Design a C program that compares two structures and displays whether their contents are equal.
Unit 03: Pointers	13	Define a pointer. Write the syntax for declaring a pointer and state any two uses of pointers in C.
	14	Explain how the address of a variable is accessed in C. Illustrate with a simple example using the address-of (&) operator.
	15	Write a C program to declare and initialize a pointer and access the value of a variable through its pointer.
	16	Analyze how pointer increment works in C. Explain the concept of scale factor with an example involving different data types.
	17	Compare pointers and arrays in C. Justify when pointers are preferred over arrays in program design.
	18	Design a C program using pointers to pass an array of structures to a function and display the contents.

Unit 04: Dynamic Memory Allocation, Pre-processor and Debugging	19	Define Dynamic Memory Allocation and list any four dynamic memory allocation functions used in C.
	20	Explain the role of the C preprocessor. Describe macro substitution, file inclusion, and compiler control directives with suitable examples.
	21	Analyze the differences between malloc() and calloc() with respect to memory allocation and initialization.
	22	Write a program to demonstrate the use of compiler control directives.
	23	Describe the role of file inclusion in C programs.
	24	Explain the difference between static memory allocation and dynamic memory allocation.
Unit 05: OOPS Concepts (C++):	25	Define Class and Object.
	26	Compare and analyze abstract classes and normal classes, highlighting their use cases.
	27	Explain the concept of abstract classes with an example.
	28	Explain inheritance in OOP. Describe its types and state its advantages.
	29	Write a program to demonstrate the use of class, object, constructor, and destructor.
	30	Write a program to demonstrate inheritance in object-oriented programming.